Name	Shyane Novelle R. Canayan	Date	October 27, 2026
Course & Year	BSCA 3rd Year		

Lab Activity 2

CPU Scheduling Algorithms

Introduction

CPU scheduling is an operating system function that determines which process in the ready queue receives CPU time. Different scheduling algorithms use different rules to determine execution order, affecting waiting time, turnaround time, response time, and overall system responsiveness.

In this laboratory activity, students will develop a CPU Scheduling Simulator using Python in GitHub Codespaces. They will implement and compare five algorithms: First-Come, First-Served (FCFS), non-preemptive Shortest Job First (SJF), non-preemptive Priority Scheduling, Round Robin (RR), and preemptive Shortest Remaining Time First (SRTF).

Learning Objectives

At the end of the activity, students should be able to:

- Create and manage a CPU scheduling laboratory repository using GitHub.
- Use GitHub Codespaces to develop and execute Python programs.
- Implement five CPU scheduling algorithms.
- Generate Gantt charts showing process execution.
- Calculate completion time, turnaround time, waiting time, and response time.
- Compare the performance of scheduling algorithms.
- Explain preemption, time quantum, starvation, and context-switching overhead.
- Commit and synchronize source code using GitHub.

Required Tools

- Existing GitHub account
- Web browser and internet connection
- GitHub Codespaces

- Python 3 environment

Dataset

Use the following dataset for FCFS, SJF, Priority Scheduling, and Round Robin.

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	0	4	1
P3	0	2	4
P4	0	6	3

Scheduling assumptions:

- A smaller priority number indicates a higher priority.
- Round Robin uses a time quantum of 2 ms.
- SJF and Priority Scheduling are non-preemptive.
- FCFS uses arrival order, with process ID breaking arrival-time ties.
- For SRTF, use the separate dataset below to demonstrate preemption.

Separate SRTF dataset:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	1	4	1
P3	2	2	4
P4	3	1	3

Step-by-Step Procedure Using GitHub

Part A. Create the GitHub Repository

1. Sign in to your existing account at https://github.com.
2. Click the + icon in the upper-right corner and select New repository.
3. Enter repository name: OS-Laboratory-2-CPU-Scheduling.
4. Enter description: Operating Systems Lab 2 - CPU Scheduling Algorithms.
5. Choose Private or Public.
6. Check Add a README file, then click Create repository.
7. In the repository, click Code, open the Codespaces tab, and select Create codespace on main.
8. Wait for the browser-based development environment to load.

9. In Codespaces, locate the Explorer panel on the left.
10. Click New File and name it cpu_scheduling.py.
11. Open the file and enter the complete program below.
12. Save the file using Ctrl+S (Windows) or File > Save.

```
# Operating Systems Laboratory Activity 2
# CPU Scheduling Simulator - Python 3

from collections import deque
from copy import deepcopy

PROCESSES = [
    {"pid": "P1", "at": 0, "bt": 8, "priority": 2},
    {"pid": "P2", "at": 0, "bt": 4, "priority": 1},
    {"pid": "P3", "at": 0, "bt": 2, "priority": 4},
    {"pid": "P4", "at": 0, "bt": 6, "priority": 3},
]

SRTF_PROCESSES = [
    {"pid": "P1", "at": 0, "bt": 8, "priority": 2},
    {"pid": "P2", "at": 1, "bt": 4, "priority": 1},
    {"pid": "P3", "at": 2, "bt": 2, "priority": 4},
    {"pid": "P4", "at": 3, "bt": 1, "priority": 3},
]

def prepare(processes):
    result = deepcopy(processes)
    for p in result:
        p["remaining"] = p["bt"]
        p["first_start"] = None
    return result

def record_run(p, start, end, gantt):
    if start == end:
```

```
        return
    if gantt and gantt[-1][2] == p["pid"] and gantt[-1][1] == start:
        gantt[-1] = (gantt[-1][0], end, p["pid"])
    else:
        gantt.append((start, end, p["pid"]))
    if p["first_start"] is None:
        p["first_start"] = start
    p["remaining"] -= end - start
    if p["remaining"] == 0:
        p["ct"] = end

def finish_results(processes):
    for p in processes:
        p["tat"] = p["ct"] - p["at"]
        p["wt"] = p["tat"] - p["bt"]
        p["rt"] = p["first_start"] - p["at"]

def fcfs(data):
    ps = prepare(data)
    time, gantt = 0, []
    for p in sorted(ps, key=lambda x: (x["at"], x["pid"])):
        if time < p["at"]:
            time = p["at"]
        start = time
        time += p["bt"]
        record_run(p, start, time, gantt)
    finish_results(ps)
    return ps, gantt

def non_preemptive(data, mode):
    ps = prepare(data)
    time, done, gantt = 0, set(), []
    while len(done) < len(ps):
        ready = [p for p in ps if p["pid"] not in done and p["at"] <= time]
        if not ready:
            time = min(p["at"] for p in ps if p["pid"] not in done)
```

continue

```
        if mode == "sjf":
            chosen = min(ready, key=lambda p: (p["bt"], p["at"], p["pid"]))
        else:
            chosen = min(ready, key=lambda p: (p["priority"], p["at"], p["pid"]))
        start = time
        time += chosen["bt"]
        record_run(chosen, start, time, gantt)
        done.add(chosen["pid"])
    finish_results(ps)
    return ps, gantt

def round_robin(data, quantum=2):
    if quantum <= 0:
        raise ValueError("Time quantum must be greater than zero.")
    ps = prepare(data)
    by_arrival = sorted(ps, key=lambda p: (p["at"], p["pid"]))
    ready, gantt = deque(), []
    time, i = 0, 0
    while i < len(by_arrival) or ready:
        if not ready:
            time = max(time, by_arrival[i]["at"])
        while i < len(by_arrival) and by_arrival[i]["at"] <= time:
            ready.append(by_arrival[i])
            i += 1
        p = ready.popleft()
        start = time
        end = time + min(quantum, p["remaining"])
        record_run(p, start, end, gantt)
        time = end
        while i < len(by_arrival) and by_arrival[i]["at"] <= time:
            ready.append(by_arrival[i])
            i += 1
        if p["remaining"] > 0:
            ready.append(p)
    finish_results(ps)




    return ps, gantt
```

```
def srtf(data):
    ps = prepare(data)
    time = min(p["at"] for p in ps)
    completed, gantt = set(), []
    while len(completed) < len(ps):
        ready = [
            p for p in ps
            if p["at"] <= time and p["pid"] not in completed and p["remaining"] > 0
        ]
        if not ready:
            time = min(p["at"] for p in ps if p["pid"] not in completed and p["at"] > time)
            continue
        p = min(ready, key=lambda x: (x["remaining"], x["at"], x["pid"]))
        start, end = time, time + 1
        record_run(p, start, end, gantt)
        time = end
        if p["remaining"] == 0:
            completed.add(p["pid"])
    finish_results(ps)
    return ps, gantt

def display(title, processes, gantt):
    print("\\n" + "=" * 65)
    print(title)
    print("=" * 65)
    print("\\nGANTT CHART")
    print(" | ".join(f"{pid} [{start}-{end}]" for start, end, pid in gantt))
    print("\\nPROCESS METRICS")
    print(f"{'PID':<6}{'AT':>5}{'BT':>5}{'PR':>5}{'CT':>6}{'TAT':>6}{'WT':>6}{'RT':>6}")
    for p in sorted(processes, key=lambda x: x["pid"]):

print(f"{p['pid']:<6}{p['at']:>5}{p['bt']:>5}{p['priority']:>5}{p['ct']:>6}{p['tat']:>6}{p['wt']:>6}{p['rt']:>6}")
    n = len(processes)
    print(f"\\nAverage Waiting Time: {sum(p['wt'] for p in processes) / n:.2f} ms")
    print(f"Average Turnaround Time: {sum(p['tat'] for p in processes) / n:.2f} ms")




print(f"Average Response Time: {sum(p['rt'] for p in processes) / n:.2f} ms")
```

```
if __name__ == "__main__":
    ps, g = fcfs(PROCESSES)
    display("1. FCFS", ps, g)
    ps, g = non_preemptive(PROCESSES, "sjf")
    display("2. NON-PREEMPTIVE SJF", ps, g)
    ps, g = non_preemptive(PROCESSES, "priority")
    display("3. NON-PREEMPTIVE PRIORITY", ps, g)
    ps, g = round_robin(PROCESSES, quantum=2)
    display("4. ROUND ROBIN (QUANTUM = 2)", ps, g)
    ps, g = srtf(SRTF_PROCESSES)
    display("5. SRTF (SEPARATE ARRIVAL-TIME DATASET)", ps, g)
```

Save the program

Press Ctrl + S on Windows, or choose File → Save.

Part C. Run the Program

13. In Codespaces, select Terminal > New Terminal.
14. Run python3 cpu_scheduling.py. If unavailable, try python cpu_scheduling.py.
15. Examine the Gantt chart, process metrics, and averages for each algorithm.
16. Take a screenshot of the terminal output for your report.

Reference Results

For the first four algorithms, the average metrics should match these values:

Algorithm	Average WT	Average TAT	Average RT
FCFS	8.50 ms	13.50 ms	8.50 ms
Non-preemptive SJF	5.00 ms	10.00 ms	5.00 ms
Non-preemptive Priority	8.50 ms	13.50 ms	8.50 ms
Round Robin (q = 2)	9.00 ms	14.00 ms	3.00 ms

SRTF uses the separate dataset because its processes have different arrival times.

Part D. Verify the Results Manually

17. Draw a Gantt chart for each algorithm and check the execution order.
18. Verify these orders for the original dataset: FCFS: P1 → P2 → P3 → P4; SJF: P3 → P2 → P4 → P1; Priority: P2 → P1 → P4 → P3.
19. For Round Robin, verify that each ready process receives up to 2 ms per turn and unfinished processes return to the end of the ready queue.
20. For SRTF, identify where the running process is preempted when another process has a shorter remaining burst time.
21. Manually calculate at least two processes for every algorithm and compare your answers with the program.

Use these formulas:

Metric	Formula
Turnaround Time (TAT)	TAT = CT − AT
Waiting Time (WT)	WT = TAT − BT
Response Time (RT)	RT = First CPU Start − AT
Average metric	Sum of metric values ÷ number of processes

Example: In FCFS, P2 starts at 8 ms and finishes at 12 ms. Thus CT = 12, TAT = 12 − 0 = 12 ms, WT = 12 − 4 = 8 ms, and RT = 8 − 0 = 8 ms.

Gantt Charts

FCFS

P1 [0-8] | P2 [8-12] | P3 [12-14] | P4 [14-20]

Non-preemptive SJF

P3 [0-2] | P2 [2-6] | P4 [6-12] | P1 [12-20]

Non-preemptive Priority

P2 [0-4] | P1 [4-12] | P4 [12-18] | P3 [18-20]

Round Robin (q = 2)

P1 [0-2] | P2 [2-4] | P3 [4-6] | P4 [6-8] | P1 [8-10] | P2 [10-12] | P4 [12-14] | P1 [14-16] | P4 [16-18] | P1 [18-20]

SRTF (Separate Dataset)

P1 [0-1] | P2 [1-2] | P3 [2-4] | P4 [4-5] | P2 [5-8] | P1 [8-15]

Calculation Tables (Process Metrics)

FCFS

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	8	8	0	0
P2	0	4	1	12	12	8	8
P3	0	2	4	14	14	12	12
P4	0	6	3	20	20	14	14

Non-preemptive SJF

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	20	20	12	12
P2	0	4	1	6	6	2	2
P3	0	2	4	2	2	0	0
P4	0	6	3	12	12	6	6

Non-preemptive Priority

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	12	12	4	4
P2	0	4	1	4	4	0	0
P3	0	2	4	20	20	18	18
P4	0	6	3	18	18	12	12

Round Robin (q = 2)

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	20	20	12	0
P2	0	4	1	12	12	8	2
P3	0	2	4	6	6	4	4
P4	0	6	3	18	18	12	6

SRTF

PID	AT	BT	PR	CT	TAT	WT	RT
P1	0	8	2	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	4	4	2	0	0
P4	3	1	3	5	2	1	1

Baseline

Comparison Table

Algorithm	Average WT	Average TAT	Average RT
FCFS	8.50 ms	13.50 ms	8.50 ms
SJF	5.00 ms	10.00 ms	5.00 ms
Priority	8.50 ms	13.50 ms	8.50 ms
Round Robin (q=2)	9.00 ms	14.00 ms	3.00 ms
SRTF	2.75 ms	6.50 ms	0.25 ms

Part E. Experiment with Different Parameters

Step 22. Change the Round Robin Time Quantum

Find this line:

```
ps, g = round_robin(PROCESSES, quantum=2)
```

Change the quantum to 4:

```
ps, g = round_robin(PROCESSES, quantum=4)
```

Save and rerun the program. Then change the quantum to 1 and run it again. Record the average waiting time, turnaround time, and response time.

Time Quantum	Average WT	Average TAT	Average RT	CPU Allocations
q = 1	9.50 ms	14.50 ms	1.50 ms	19
q = 2 (Baseline)	9.00 ms	14.00 ms	3.00 ms	10
q = 4	9.00 ms	14.00 ms	5.50 ms	6

- How does the time quantum affect the number of CPU allocations?

As the time quantum increases, the number of CPU allocations (and context switches) decrease. A smaller quantum splits execution into more frequent, smaller time slices, resulting in more allocations.

- Which quantum produces the lowest average waiting time in your tests?

Both q = 2 and q = 4 produced the lowest average waiting time (9.00 ms) for this specific dataset. A quantum of 1 resulted in a slightly higher average waiting time (9.50 ms).

- What happens when the quantum becomes very large?

When the time quantum becomes larger than or equal to the maximum process burst time, Round Robin behaves exactly like the First-Come, First-Served (FCFS) scheduling algorithm, completing each process before switching to the next.

- Why can a very small quantum increase context-switching overhead in a real operating system?

A very small quantum causes the CPU to rapidly and continuously interrupt Running processes. The CPU spends a significant percentage of its time executing the context switch (saving process states to PCBs and loading the next state) rather than Performing actual computational work, which degrades overall system performance

Step 23. Modify the Process Dataset

Change burst times and priorities, then rerun the program. Use the same modified dataset for FCFS, SJF, Priority Scheduling, and Round Robin to compare them fairly. Keep the SRTF dataset separate unless you deliberately change its arrival times.

Note: Modified Dataset utilized for this run:
P1 (BT=5, PR=3), P2 (BT=3, PR=1), P3 (BT=4, PR=4), P4 (BT=2, PR=2).

Algorithm	Average WT	Average TAT	Average RT
FCFS	6.25 ms	9.75 ms	6.25 ms
SJF	4.00 ms	7.50 ms	4.00 ms
Priority	4.50 ms	8.00 ms	4.50 ms
Round Robin	8.00 ms	11.50 ms	3.00 ms
SRTF	3.50 ms	7.00 ms	2.25 ms

Part F. Save Your Work to GitHub

22. Click Source Control in the left sidebar.
23. Review the changed file cpu_scheduling.py.
24. Enter commit message: Implement CPU scheduling algorithms.
25. Click Commit and confirm if prompted.
26. Click Sync Changes or the available push option to upload your commit to GitHub.
27. Open your repository on GitHub and verify that the latest cpu_scheduling.py file is present.

Questions

28. What is CPU scheduling, and why is it necessary?

    CPU scheduling is an operating system mechanism that determines which process in the ready queue gets access to the CPU for execution. It is necessary to maximize CPU utilization, increase system throughput, ensure fairness among running programs, and minimize waiting and response times in multi-tasking environments.

29. What is the difference between FCFS and SJF?

    FCFS executes processes non-preemptively in the exact order they arrive in the ready queue. SJF selects the process with the smallest execution/burst time (BT) next.

30. Why can SJF produce a lower average waiting time than FCFS?

SJF minimizes waiting time by executing shorter processes first. This clears multiple short tasks quickly from the ready queue before running longer tasks, avoiding the Convoy Effect seen in FCFS.

31. How does Priority Scheduling determine the next process?

Priority Scheduling assigns a priority numerical rank to each process. The CPU selects the process with the highest priority (e.g., Priority 1 is treated higher than Priority 2).

32. What is starvation? Which algorithms can experience it?

Starvation occurs when a process is perpetually delayed from getting CPU execution time because higher-priority or shorter processes continually enter the ready queue. Non-preemptive SJF, Priority Scheduling, and SRTF can experience it.

33. How does Round Robin improve responsiveness for interactive processes?

Round Robin assigns a fixed time quantum to each process in a cyclic order. Frequent context-switching ensures every process gets quick initial access to the CPU, drastically lowering the Average Response Time.

34. What is the effect of changing the Round Robin time quantum?

A very large time quantum makes Round Robin behave like FCFS. A very small time quantum decreases response time but increases CPU context-switching overhead, reducing overall throughput.

35. How does SRTF differ from non-preemptive SJF?

SJF runs a process to completion once started. SRTF is the preemptive version; if a new process arrives with a shorter remaining burst time than the running process, the OS interrupts the current process to run the new one.

36. Why do preemptive algorithms require the operating system to save and restore process context?

    When a process is preempted, the OS must save its current state (registers, program counter) into its Process Control Block (PCB). When rescheduled, the OS restores this state, so execution resumes from the exact interrupted instruction.

37. Does the algorithm with the lowest average waiting time always provide the best user experience? Explain.

    No. Algorithms like SJF or SRTF provide the lowest average waiting time but can cause starvation for longer processes. Round Robin often provides a better user experience for interactive systems because it guarantees responsiveness and prevents UI freezing.

38. Why is it important to use the same dataset when comparing algorithms?

    Using the same dataset controls external variables (arrival time, burst time, priority) so that performance differences reflect the inherent design of the scheduling algorithm rather than variations in workload.

39. What role does GitHub play in managing and submitting your laboratory work?

    GitHub provides version control to track source code changes, cloud hosting for the project repository, integration with Code spaces for browser-based development, and a centralized platform for submitting lab deliverables.

Conclusions

Analysis:

1. SJF achieved the lowest average waiting time (5.00 ms) among non-preemptive algorithms on the baseline dataset.
2. Round Robin (q=2) yielded the best response time (3.00 ms) on the baseline dataset, confirming its effectiveness for interactive environments.
3. SRTF achieved the lowest overall average waiting time (2.75 ms) and response time (0.25 ms) across all tested algorithms due to its preemptive evaluation of arrival times.

Conclusion:

CPU scheduling algorithms represent a trade-off between system throughput, fairness, and responsiveness. Non-preemptive algorithms like FCFS and SJF minimize context-switching overhead but suffer from convoy effects or potential starvation. Preemptive algorithms like Round Robin and SRTF optimize responsiveness and waiting times at the cost of context-

switching management.

GitHub Codespaces successfully served as an environment for simulating and evaluating these operating system mechanisms.

Required Deliverables

- GitHub repository link
- Completed Python source code
- Screenshot of the program running in Codespaces
- Gantt charts for all five algorithms
- Calculation tables showing CT, TAT, WT, and RT
- Comparison table of average metrics
- Results of the Round Robin time-quantum experiment
- Answers to the laboratory questions
- Brief analysis and conclusion